

Research Map

The MCP Server

How any AI assistant can do research **directly inside our app** - and what happens behind the scenes

In one sentence: you tell an AI assistant such as Gemini CLI *"research cloud computing"*, and while it searches the web, a new workspace opens in Research Map by itself and fills up live with the pages it reads, the questions it asks, the conclusions it draws and the connections between them - all in a separate **AI Agent branch** so a human reviews it before it becomes part of the main research.

What	Answer
Feature	MCP server built into our Go backend (endpoint <code>/api/v1/mcp</code>)
Protocol	Model Context Protocol (MCP), "Streamable HTTP" transport, JSON-RPC 2.0 messages
Works with	Gemini CLI, Claude Code, Claude Desktop, Cursor, Cline - any MCP-compatible client
Tools offered	13 tools (<code>start_research</code> , <code>add_source</code> , <code>add_finding</code> , <code>search_workspace</code> , ...)
Key libraries	Official MCP Go SDK v1.8.0, Gin, pgx, coder/websocket, FastAPI, traefik, fastembed
Safety	Personal tokens (hashed), role checks, AI Agent branch, rate limit, SSRF guard

1. What is MCP? (the simple version)

AI assistants are good at thinking and searching, but on their own they cannot use *your* apps. **MCP (Model Context Protocol)** is an open standard, created by Anthropic and now supported by Google, OpenAI and others, that lets an app offer **tools** to any AI assistant in one common format.

Think of it like **USB-C for AI**: we build one MCP server once, and every assistant that "speaks MCP" can plug into it - no special integration for each AI company.

- **MCP server** = our app. It says: "here are my tools, here is what each one needs".
- **MCP client** = the AI assistant (Gemini CLI, Claude ...). It reads the tool list and decides when to call a tool.
- **Tool** = one action, like `add_source(url, title, content)`. Inputs and outputs are described with JSON Schema so the AI knows exactly what to send.
- **Transport** = how messages travel. We use **Streamable HTTP**: normal HTTP POST requests with JSON-RPC 2.0 messages, so it works through any network and needs no extra server.

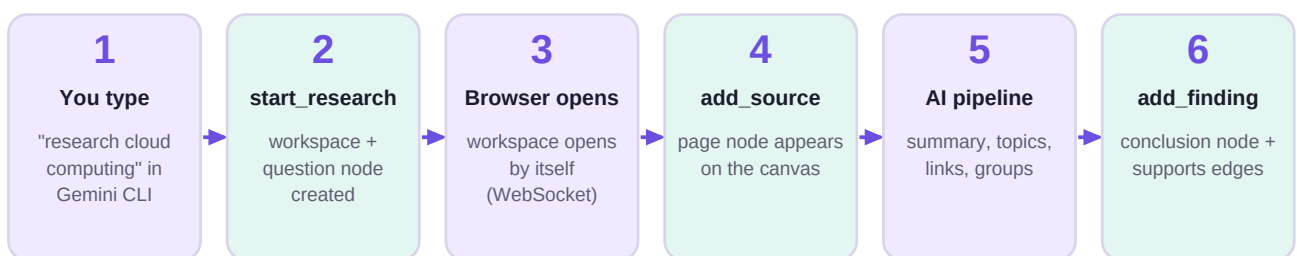
2. What our MCP feature does

Our product turns research into a visual **knowledge map**: pages, questions, findings and explained connections on a canvas. Normally our Chrome extension feeds it while a human browses. The MCP server adds a second way in: **an AI assistant can research for you and its work appears on the same map**, live.

Demo in one minute:

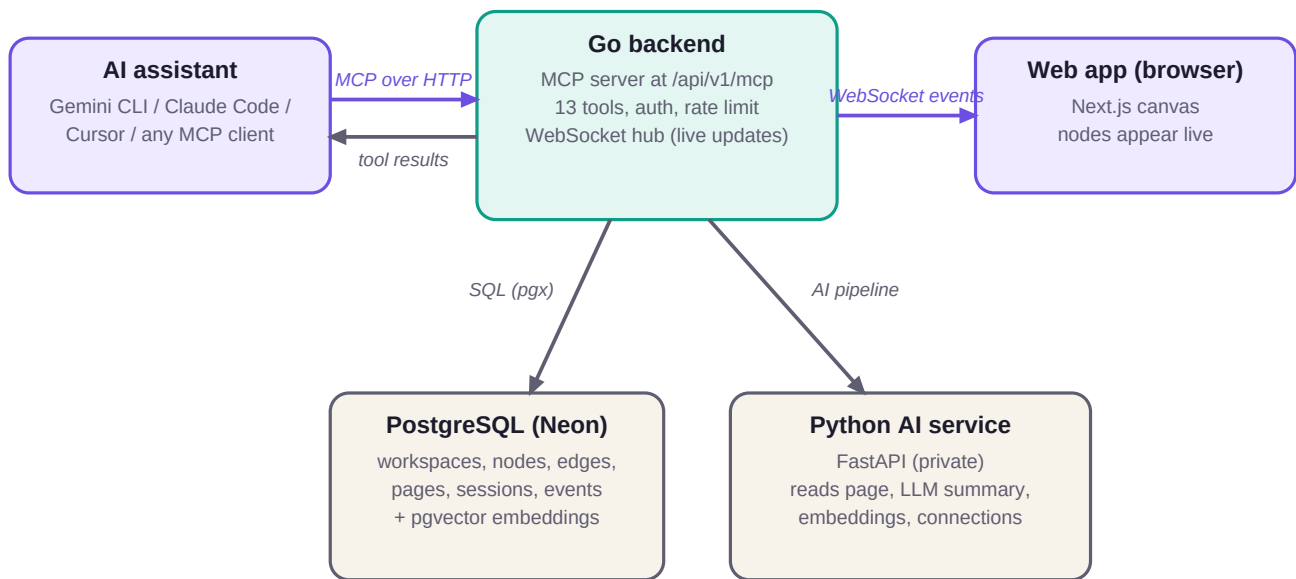
- Open Research Map in the browser (any page, even the empty dashboard).
- In Gemini CLI type: *research cloud computing using research-canvas*.
- Gemini calls **start_research** - the browser jumps into a brand-new workspace named after the topic.
- Every page Gemini reads becomes a node; our AI reads the full page, writes a summary, picks topics and draws connections.
- Gemini's conclusions appear as **Finding** nodes linked to the sources that support them.

The six steps you see on screen



Steps 4-6 repeat for every page the assistant reads, so the map grows while it researches.

3. The big picture (architecture)



The assistant never touches the database or the browser directly: everything goes through the Go backend.

Part	Role in the MCP feature	Built with
AI assistant	Searches the web itself and calls our tools with what it found.	Gemini CLI 0.62 (or Claude Code, Cursor ...)
Go backend	The MCP server. Checks the token, runs each tool, saves to the database, broadcasts live events. The only part that talks to PostgreSQL.	Go 1.26, Gin, official MCP Go SDK
Python AI service	Private helper only Go can call. Downloads and cleans the page, asks the LLM for a summary, topics and claims, creates embeddings, suggests connections.	FastAPI, trafilatura, Groq / Gemini LLMs, fastembed
PostgreSQL (Neon)	Stores everything: workspaces, branches, nodes, edges, pages, sessions, tokens, events. pgvector stores embeddings for "similar meaning" search.	PostgreSQL + pgvector + pg_trgm
Web app	Shows the canvas. Listens on a WebSocket and adds nodes the moment the server announces them.	Next.js 16, React 19, React Flow, Zustand

4. Behind the scenes, step by step

Step 0 - Connecting (once, when the assistant starts)

- The user creates a personal **MCP token** in Settings -> AI Assistant (MCP) and pastes it into the assistant's config (for Gemini CLI: `~/ .gemini/settings.json`). The assistant sends it as `Authorization: Bearer rm_mcp_...`
- The assistant sends **initialize**. Our server answers with its name, version and **instructions** ("call `start_research` first, then `add_source` for every page ...").
- The assistant asks **tools/list** and gets our 13 tools with JSON-Schema descriptions of their inputs and outputs.
- The server gives the connection a session id (`Mcp-Session-Id`) and remembers which user owns it.

Step 1 - `start_research(topic)`

- Creates a new workspace named after the topic (or reuses one with the same name, so a retry never makes duplicates) with three branches: **Main**, the user's personal branch and the **AI Agent** branch.
- Adds the topic as a **Question** node and starts an "AI assistant" research session (used for the numbered reference list).
- Sends the event `mcp.research_started` to **every open tab of that user** over WebSocket. The web app hears it and navigates into the new workspace - this is why the browser opens by itself.
- Remembers this workspace for the rest of the MCP session, so later tools do not even need a workspace id.
- **Background sources:** the server also searches Wikipedia's free public API for the topic and adds the top 4 articles as sources (labeled "Research Map (Wikipedia)"), so the map fills with real, analyzed sources within seconds even if the assistant's own web search is slow or rate-limited.

Step 2 - `add_source(url, title, content, question)`

- Checks: valid http(s) URL, not on the workspace's blocklist, user is an editor or owner.
- Links the page to the Question node of the search that found it, saves the page and creates a page node with status *analyzing* in the AI Agent branch - all in one database transaction (with a lock so the same URL is never added twice).
- Answers the assistant immediately with the new `node_id`, then in the background broadcasts `page.created + node.created`: the node pops onto the canvas.
- Puts the node into the **AI pipeline queue** (next section).

Step 3 - the AI pipeline (runs in the background for each source)

Stage	What happens	Tech
Agent 1 - Understand	Assistants send only a short snippet, so the Python service downloads the real public page and extracts the main text. An LLM writes the summary, main concept, topics, claims (with exact quotes) and questions answered.	httpx + trafilatura (pypdf for PDFs), Gemini / Groq LLM with JSON output
Embeddings	Turns the summary and each claim into number vectors that capture meaning.	fastembed, BAAI/bge-small-en-v1.5, pgvector
Agent 2 - Connect	Finds the most similar existing nodes and asks the LLM how they relate (answers, explains, supports, contradicts ...) with a one-sentence reason. Saved as suggested edges.	pgvector similarity + LLM

Stage	What happens	Tech
Agent 3 - Group	Places the node into a topic group (or creates one) on the canvas.	LLM + clustering
Conflict check	Compares new claims with claims from other sources and flags possible disagreements.	embeddings + LLM

Every stage sends a WebSocket event, so the user watches the badge go from *Analyzing* to ready, and edges and groups appear.

Step 4 - add_finding(statement, details, source_node_ids)

- Creates a **Finding** node next to its sources. Its details hold the assistant's explanation plus a "Supported by" list of source titles.
- Adds a **supports** edge from every source to the finding. Ids are verified to belong to this workspace (a page id is mapped to its node; unknown ids are skipped and reported back to the assistant instead of failing).

Step 5 - the user stays in control

Everything from MCP lives in the **AI Agent branch**, marked with an "AI Agent" badge and a robot icon. The user can accept, change or reject connections, and copy the good items to **Main** - the assistant can suggest, but never decides.

5. The 13 MCP tools

`workspace_id` is optional on every tool: it defaults to the workspace from `start_research`, then to the workspace the user opened last.

Tool	What the assistant gives	What it does
<code>start_research</code>	topic	Creates/reuses the workspace, adds the Question, opens it live in the browser. Called first.
<code>add_source</code>	url, title, content, question, note	Adds a web page; runs the full AI pipeline.
<code>add_finding</code>	statement, details, source_node_ids	Adds a conclusion linked to its sources with supports edges.
<code>add_question</code>	question	Records a sub-question / search query as a Question node.
<code>add_note</code>	text, node_id (optional)	Adds a note to a node, or a free Note node.
<code>link_nodes</code>	source_id, target_id, relation, reason	Suggests an explained connection (the user accepts or rejects).
<code>list_workspaces</code>	-	Workspaces the user can access, with role.
<code>search_workspace</code>	query	Hybrid search: full text + fuzzy + meaning (pgvector).
<code>get_node</code>	node_id	Node details: summary, notes, connections with reasons.
<code>get_graph_outline</code>	-	Compact text outline: topic groups and the nodes inside.
<code>list_conflicts</code>	-	Sources that may disagree (the app never decides who is right).
<code>get_references</code>	-	Numbered reference list (Markdown) for citing.
<code>get_session_report</code>	session_id (optional)	Research statistics and the AI summary report.

6. Packages and tools we used

Package / tool	Version	Why we used it
github.com/modelcontextprotocol/go-sdk	v1.8.0	Official MCP SDK for Go. <code>mcp.NewServer</code> , <code>mcp.AddTool</code> and <code>NewStreamableHTTPHandler</code> give us a standard MCP server inside the existing backend.
github.com/google/jsonschema-go	v0.4.3	Generates the JSON Schema of each tool's input from Go structs; we relax it so LLM clients can send null/extra fields.
github.com/gin-gonic/gin	v1.12.0	HTTP router; the MCP endpoint is one route: <code>r.Any("/mcp", ...)</code> .
github.com/jackc/pgx/v5 (pgxpool)	v5.11.0	Fast PostgreSQL driver with a connection pool (kept warm for low latency).
github.com/coder/websocket	v1.8.15	WebSocket hub: one room per workspace + one per user for live updates.
github.com/golang-jwt/jwt/v5 , crypto/sha256	v5.3.1	Web login sessions; MCP tokens are stored only as SHA-256 hashes.
pressly/goose	v3.28.0	Database migrations.
FastAPI + uvicorn (Python)	>=0.115	Private AI service that Go calls for page understanding and connections.
trafilatura , pypdf , httpx	latest	Download the public page safely and extract its main text (also PDFs).

Package / tool	Version	Why we used it
fastembed (BAAI/bge-small-en-v1.5)	>=0.5	Local embeddings - no paid embedding API.
Gemini / Groq LLM APIs	-	Summaries, topics, claims, connection reasons (with automatic fallback between providers).
PostgreSQL + pgvector + pg_trgm (Neon)	-	Storage, meaning search, fuzzy search.
Next.js, React, React Flow, Zustand, TanStack Query	16 / 19 / 12 / 5 / 5	The canvas UI and its live state.
@modelcontextprotocol/sdk (TypeScript)	1.31	Used only for testing: the same client library Gemini CLI uses.
Gemini CLI	0.62.0	The AI assistant used in our demo (config: ~/.gemini/settings.json + GEMINI.md).

7. Security - why this is safe

Risk	How we handle it
Someone uses a stolen or guessed token	Tokens are 24 random bytes (rm_mcp_ . . .). Only the SHA-256 hash is stored, so even a database leak does not reveal them. Tokens can be revoked in Settings, which takes effect immediately.
A token for the extension used for MCP	Each token has a kind; the MCP endpoint accepts only <i>mcp</i> tokens.
Writing into someone else's workspace	Every tool checks membership and role. Viewers can read but never write. Node ids sent by the AI are always re-checked against the workspace.
Hijacking another user's MCP session	Each Mcp-Session-Id is bound to the user who opened it; other users get 403.
AI assistant adds wrong or junk content	Everything goes to the separate AI Agent branch , labeled "AI-suggested". Humans review and copy to Main.
Server tricked into fetching internal URLs (SSRF)	The page fetcher only allows public IP addresses, re-checks every redirect, and limits size and time.
Too many requests	Rate limit: 240 MCP requests per minute per user, then HTTP 429.
Private sites (mail, banking)	The workspace blocklist is enforced for MCP sources too.

8. Problems we hit and how we solved them

Making a real assistant work end to end taught us a lot. These are the main fixes:

Problem	Cause	Fix
Gemini CLI showed "MCP issues" and saw no tools	The SDK described each tool's output as JSON Schema <code>true</code> ; Gemini's validator requires an object, so it rejected the whole list.	Explicit output schema on every tool.
"MCP ERROR" after the first call	Models send <code>null</code> for optional fields or extra fields; the strict schema rejected them.	Lenient input schemas (null + unknown fields allowed); handlers validate what matters.
The assistant never called our tools	Every write needed a <code>workspace_id</code> it did not know.	<code>start_research</code> tool + optional <code>workspace_id</code> + clear instructions (server instructions and GEMINI.md).
Nodes did not appear without refresh	The browser was not in the new workspace, and events sent before the page loaded were missed.	User-level WebSocket event opens the workspace; the canvas re-syncs whenever its socket (re)connects.
Thin page details	Assistants send a 2-sentence snippet.	For MCP sources the AI service downloads the full page before summarizing.
Slow tool calls (3-6 s)	Remote database (~230 ms per round trip) and ~27 queries per <code>add_source</code> .	Per-session caches, cached token check, async broadcasting, warm DB pool: about 11 queries; the call returns as soon as the node is saved.
"MCP ERROR" during long research	Gemini CLI (Node.js) drops a streaming connection that receives no data for 5 minutes.	Server keep-alive: a ping every 30 s keeps the session alive (3 missed pings allowed).
Gemini stuck on its own web search	The free Gemini tier has a small quota for Google Search; the assistant waits and retries.	<code>start_research</code> also adds Wikipedia background sources server-side, so the canvas fills anyway; the assistant adds more when it can.

Problem	Cause	Fix
AI session showed as "Tracking"	The assistant's session looked like the user's own tracking session.	AI Agent sessions are excluded from the user's tracking state.

9. How to connect an assistant

1) In Research Map: **Settings -> AI Assistant (MCP) -> create token**. 2) Add the server to your assistant:

Gemini CLI - ~/.gemini/settings.json

```
{
  "mcpServers": {
    "research-canvas": {
      "url": "http://localhost:8080/api/v1/mcp", "type": "http",
      "headers": { "Authorization": "Bearer YOUR_MCP_TOKEN" }, "trust": true
    }
  }
}
```

Plus ~/.gemini/GEMINI.md with the research workflow (start_research first, add_source after every search, add_finding with details). Check with `/mcp list: research-canvas - Ready (13 tools)`.

Claude Code

```
claude mcp add --transport http research-canvas http://localhost:8080/api/v1/mcp
--header "Authorization: Bearer YOUR_MCP_TOKEN"
```

Cursor / Cline / others - the same JSON block (url + headers) in their MCP settings.

10. How we tested it

- **End-to-end suite** (scripts/mcp_e2e.py, official MCP Python SDK): 44 checks - tools, permissions, viewer cannot write, revoked token is rejected, session ownership, live WebSocket events, AI pipeline results. **44/44 pass**.
- **Gemini's own client library** (TypeScript MCP SDK 1.31) with Gemini-style calls: nulls, extra fields, page ids instead of node ids - all accepted.
- **Real Gemini CLI run**: Gemini called start_research, the workspace opened live in the browser.
- **Browser check**: dashboard auto-opened the new workspace; question, source and finding nodes and their edges appeared without refresh.
- Go unit tests and 18 Python tests (including the SSRF guard) pass.

11. Where the code lives

File	What is inside
backend/internal/app/mcp.go	MCP server: the 13 tools, per-session workspace + caches, token auth, rate limit, session ownership.
backend/internal/app/capture.go	ingestPage: the single path that saves a page node (shared by extension, import and MCP).
backend/internal/app/pipeline.go	Background AI pipeline (Agents 1-3, conflict check) and its live events.
backend/internal/app/ws.go	WebSocket hub: workspace rooms and the per-user room.
agent/app/main.py	Python AI service: safe page fetch, understanding, embeddings, connections.
frontend/features/workspace/AgentResearchListener.tsx	Listens for mcp.research_started and opens the workspace.
frontend/features/workspace/useWorkspace.ts	Applies live events to the canvas; re-syncs on reconnect.

Summary for judges: we implemented the open MCP standard with the official SDK inside our existing Go backend, so any AI assistant can research into Research Map. The assistant does the searching; our pipeline does the understanding, connecting and organizing; the canvas shows it live; and a separate AI Agent branch keeps the human in charge.